

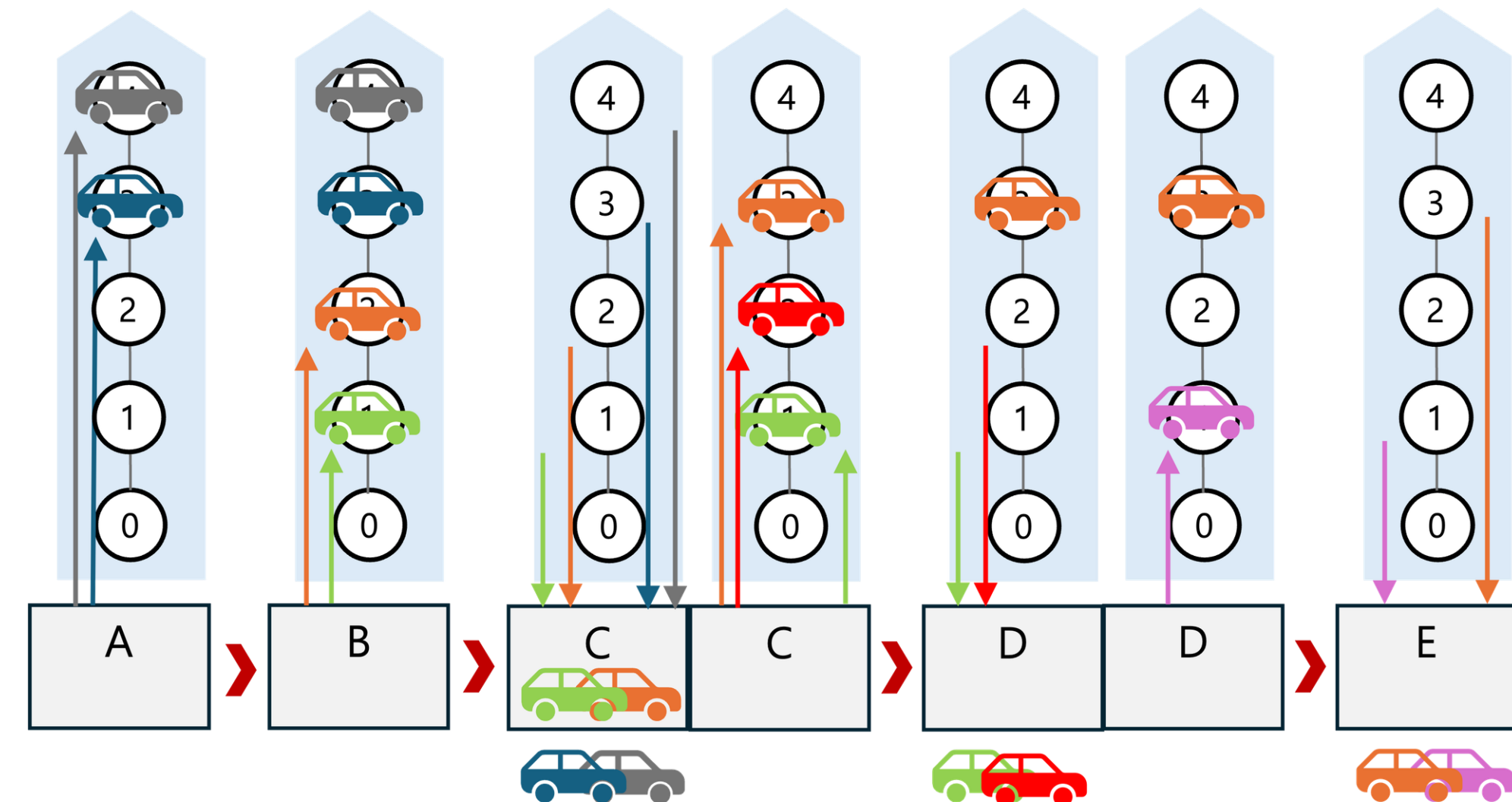
최적화와 계산지능

Roll-On/Roll-Off 선박에서의 무게 밸런싱

202127507 김민구
202127510 김범준
202427504 고병국

01. 문제 소개

■ 문제 상황 정의 및 주제 선정



기본 운항 조건 확정

차량 수송 수요: 어떤 항구에서 실어 어떤 항구에 내려야 하는지 (O-D Pair)와 각 수요에 할당된 차량 대수가 정해져 있습니다.

선박 운항 항로: 선박이 방문하는 항구의 순서가 고정되어 있습니다.



선박 운항 및 상·하역 수행

각 항구에 도착하면, 해당 항구가 출발지(O)인 차량은 **선적**(Loading) 하고, 해당 항구가 도착지(D)인 차량은 **하역**(Unloading) 하는 작업을 수행합니다.



재배치 문제 발생

방해되는 차량을 선박 내 다른 위치로 옮기거나 항구에 임시로 내렸다가 다시 싣는 작업이 필요합니다.

비용 발생: 이러한 모든 불필요한 재배치 작업은 추가 시간과 자원을 소모하므로 비용을 발생시킵니다.



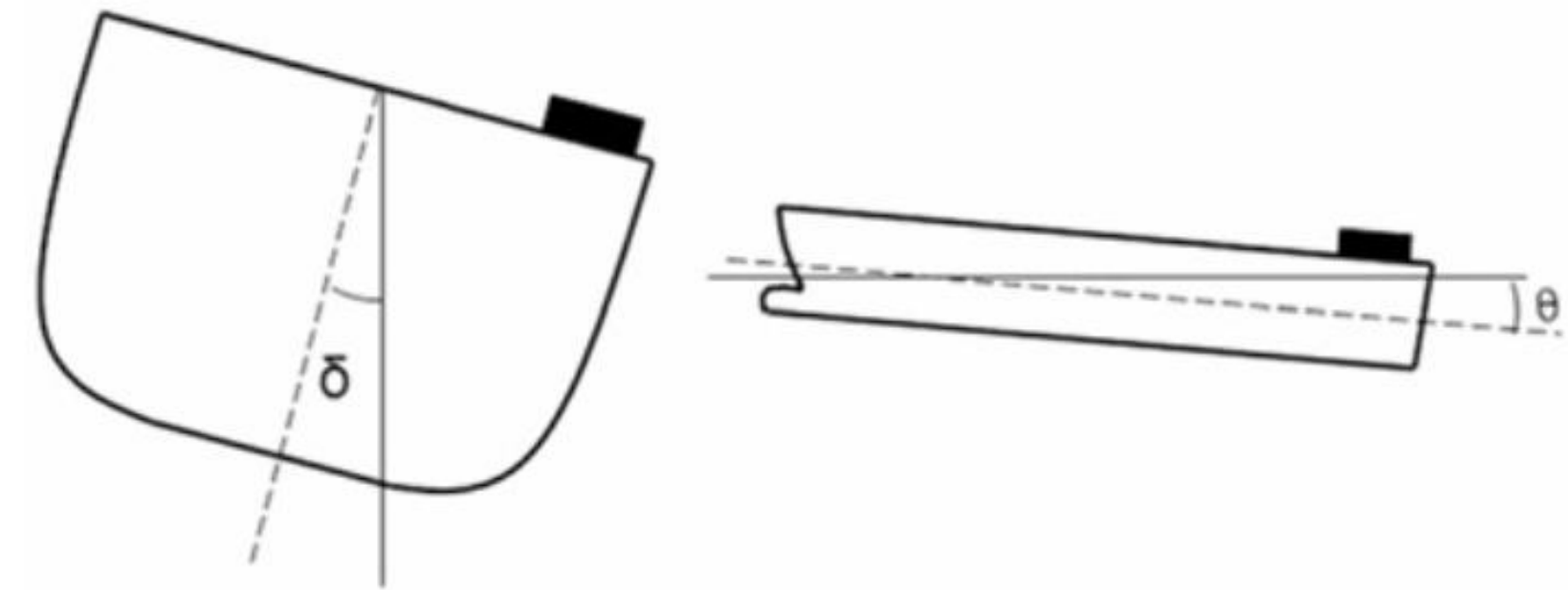
비용 최적화 계획 수립

주어진 모든 차량 수송 수요를 **100% 만족**시키는 것을 전제로 합니다.

차량을 선박에 적재하는 시점 (Step 2)부터, 전체 항해 일정 (Step 3) 동안 발생하는 '총 재배치 비용'을 최소화할 수 있는 **최적의 차량 적재 계획**을 수립해야 합니다.

```
"K_meta": [  
  {  
    "od": [  
      0,  
      1  
    ],  
    "qty": 2,  
    "class": "L",  
    "weight": 3.0  
  }  
]
```

차량 크기별 L,M,S 세가지의 Class 부여
Class별 무게 차등 부여.



차량의 다양화: 세단, SUV, 그리고 상용 트럭
차량 종류의 구분: 선적 수요는 이제 '세단', 'SUV',
'픽업트럭', '상용 버스' 등 구체적인 차종으로 구분됩
니다.
차종별 무게의 차별화: 각 차량 종류는 명확히 다른
무게를 가집니다. (예: 세단 1.5톤, SUV 2.5톤, 트럭
5톤)

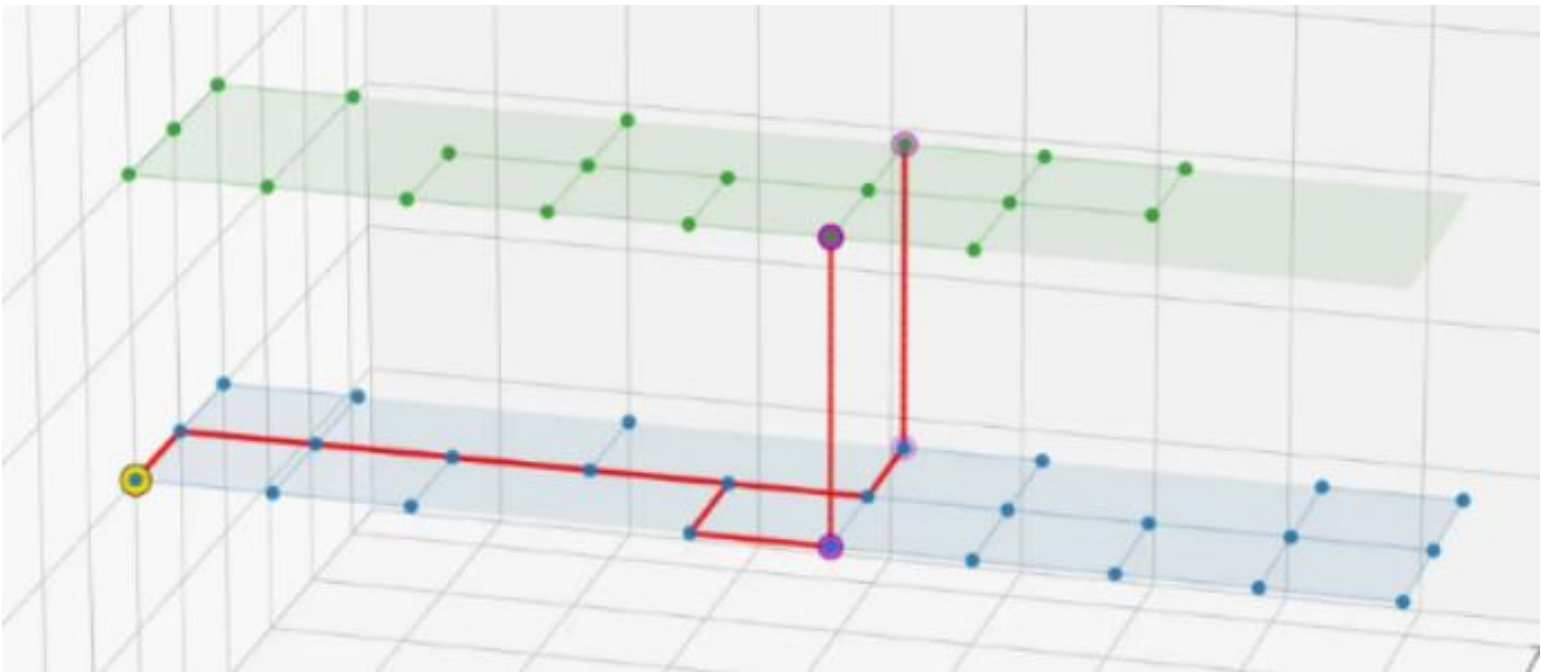
선박의 한계: 단순한 공간이 아닌 '무게'의 한계
층별 한계 하중: 선박의 각 층은 견딜 수 있는 단위 면
적당 하중 또는 총 하중 한계가 정해져 있습니다. 한 구
역에 무거운 상용 트럭 여러 대를 집중시킬 수 없습니
다.
무게 중심: 선박의 안전에 가장 치명적인 요소입니다.
선박은 무게 중심을 낮게 유지하여 복원력을 확보해야
합니다.

기존 Solution 생성

```
{
  "obj": 187.0,
  "feasible": true,
  "infeasibility": null,
  "solution": {
    "0": [
      [0, 1, 4, 7, 8, 10, 11, 13, 16, 22, 29, 36], 2
    ],
    [
      [0, 1, 4, 7, 8, 10, 11, 13, 16, 22, 29], 2
    ],
    [
      [0, 1, 4, 7, 8, 10, 12, 14, 17, 24, 30], 2
    ],
    [
      [0, 1, 4, 7, 8, 10, 12, 14, 17, 24], 2
    ],
    [
      [0, 1, 4, 7, 8, 10, 11, 13, 16, 23], 2
    ]
  ]
}
```

하나의 차량이 지나가는 노드 경로

순서화 된 해 집합
선적 시 마지막 노드는 점유되며
하역 시 마지막 노드는 0번 노드



- 1. 각각의 차량은 이미 점유된 노드를 지날 수 없음.
- 2. 각각의 차량이 노드를 지날 때 변동비 1이 발생
- 3. 각각의 차량이 재배치 발생시 변동비 100이 발생

Cost 최소화 Solution 생성시 각 차량의 무게를 무시한다!

■ 문제 인코딩 방법

```
{
  "od": [
    0,
    9
  ],
  "class": "L",
  "weight": 3.0,
  "qty": 3,
  "orig_idx": 6,
  "orig_qty": 10
},
{
  "od": [
    0,
    9
  ],
  "class": "M",
  "weight": 2.0,
  "qty": 3,
  "orig_idx": 6,
  "orig_qty": 10
},
{
  "od": [
    0,
    9
  ],
  "class": "S",
  "weight": 1.0,
  "qty": 4,
  "orig_idx": 6,
  "orig_qty": 10
},
}
```

```
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 241, 268, 288, 306, 325, 335, 348, 356, 362], 20
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 241, 268, 288, 306, 325, 335, 348, 356], 20
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 241, 268, 288, 306, 325, 335, 348], 20
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 241, 268, 288, 306, 325, 335], 19
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 244, 266, 287, 305, 323, 334, 346], 20
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 241, 268, 288, 306, 325], 19
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 244, 266, 287, 305, 323, 334], 18
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 244, 266, 287, 305, 323], 19
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 241, 268, 288, 306], 18
],
[
  [0, 1, 3, 6, 9, 11, 13, 16, 18, 20, 25, 31, 34, 38, 43, 46, 53, 65, 75, 91, 108, 135, 160, 189, 218, 244, 266, 287, 305, 324], 18
],
]
```

같은 수요 K를 무게 단위에 따라 3가지로 분리
Ex) O:port 2 / D: port 6인 수요 7번에 대하여
수요 20 : (7,S) 수요 19 : (7,M) 수요 18 : (7,L)



같은 수요별로 기존의 해에 하나의 종류로 존재하던 차량을 분리된 종류에 맞게 변환한다.

변환은 랜덤으로 이루어지며 각각의 차량 종류에 따른 개수 제약만이 존재한다.

하나의 수요는 순서에 따라 경로의 정보를 가진 하나의 리스트를 생성한다.

리스트의 리스트 G

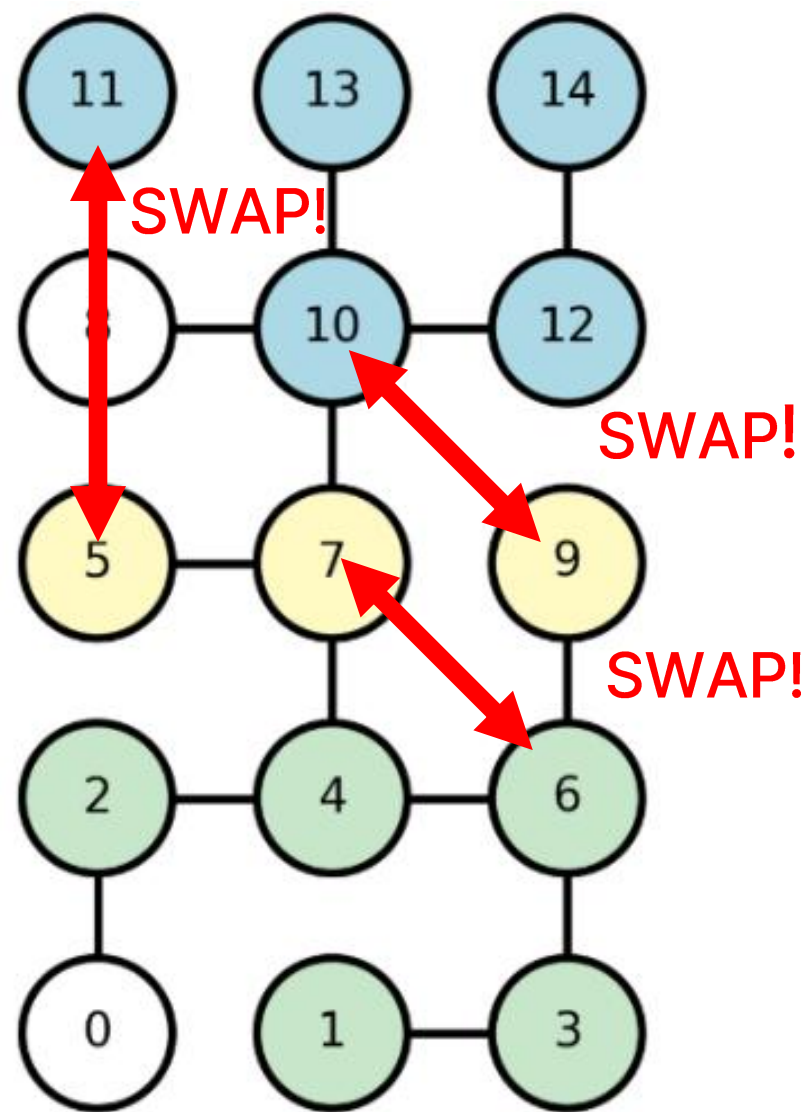
내부 리스트 (g)

$$G = [..., [S, S, S, M, S, L, M, L, L], ...]$$

(g)는 2번항구에서 6번 항구로 가는 7번 수요에 대한 리스트라고 할때 같은 S처럼 보이지만 S는 순서에 따라 경로 정보를 포함하므로 다른 객체!

하나의 L

2-opt 기반 메타휴리스틱 실험



파란색 : L
노란색 : M
초록색 : S

모두 같은 수요, 즉 출발지와 목적지가 같은 수요에 대해
다른 크기의 동일한 수요와 **SWAP**하여 **무게 밸런싱 지표** 계산!

적합도 함수 설계

Optimal stowage on Ro-Ro decks for efficiency and safety

Romanas Puisa, r.puisa@strath.ac.uk
Maritime Safety Research Centre, University of Strathclyde
Department of Naval Architecture, Ocean & Marine Engineering, Henry Dyer Building, 100 Montrose Street, Glasgow G4 0LZ

$$M_r = \sum_d \sum_g \sum_c W_g x_{dgc} \left(y_c^1 + \frac{b_c}{2} + \delta_g^y - Y_d^c + Y_d^0 \right) \tag{18}$$

$$M_t = \sum_d \sum_g \sum_c W_g x_{dgc} \left(x_c^1 + LC G_g + \delta_g^x - X_d^c + Y_d^0 \right) \tag{19}$$

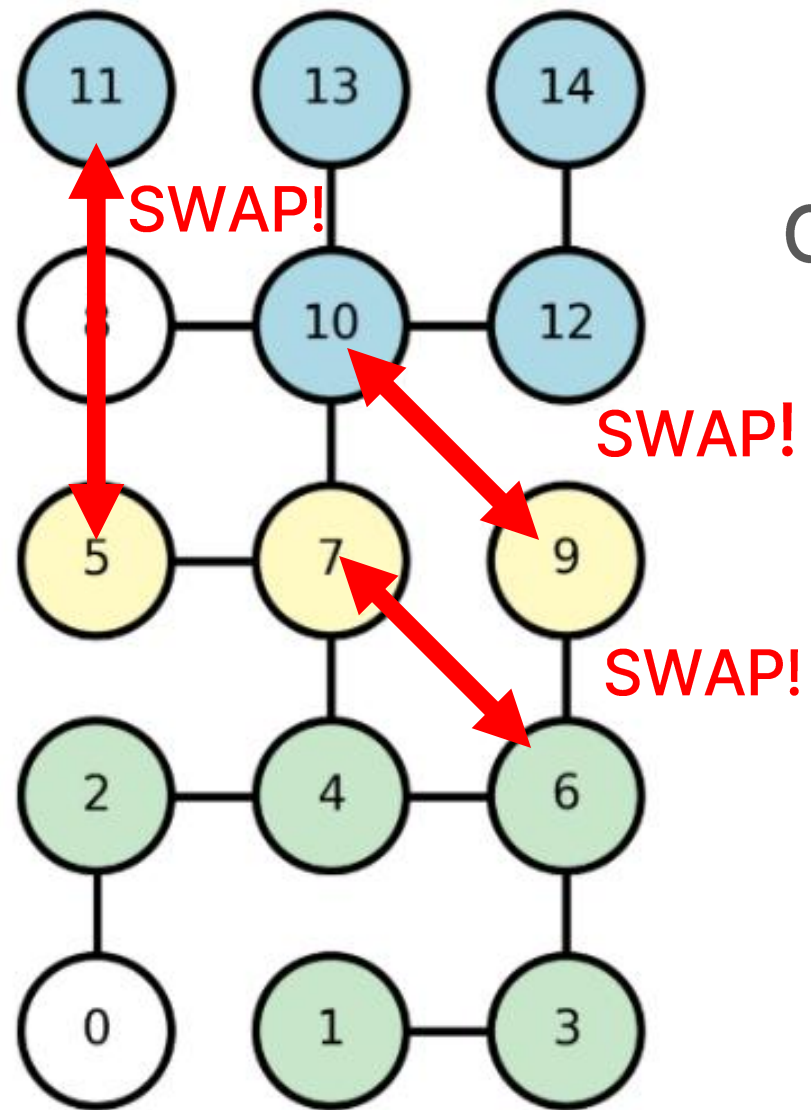
각 수요의 선적 위치와 무게를 이용하여 **모멘텀을 계산**

Puisa, R. (2021). Optimal stowage on Ro-Ro decks for efficiency and safety.
Journal of Marine Engineering & Technology, 20(1), 17–33.

$$Fitness = \frac{|CurrentMoment|}{MaxAllowableMoment}$$

각 문제상황에 따라 다를 수 있는 Fitness 값을
스케일링을 통해 균일화

SA



$G = [..., [L,L,L,L,L,M,M,M,S,S,S,S,S], ...]$

파란색 : L

노란색 : M

초록색 : S

모두 같은 수요, 즉 출발지와 목적지가 같은 수요에 대해
다른 크기의 동일한 수요와 **SWAP**하여 **무게 밸런싱 지표** 계산!

파라미터 설정

초기 온도 : $T_0 = 0.25 * E_{cur}$

냉각 스케줄 : $T_{k+1} = \alpha T_k, \alpha = 0.995$

termination 기준 : 60초 도달시 종료

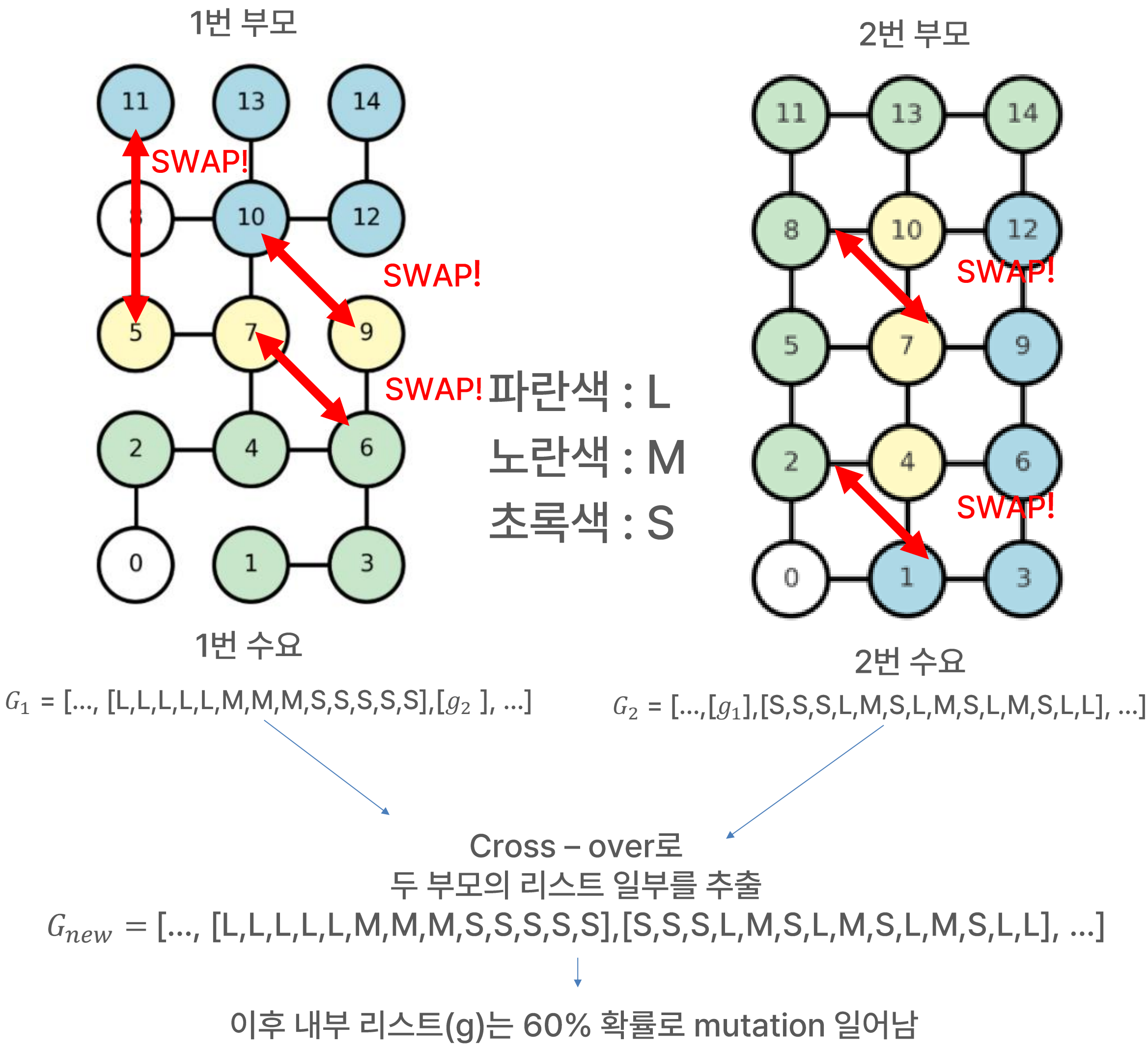
개선해 채택 방법

$$\Delta E = f_{cur} - f_{swap}$$

$\Delta E \leq 0$ 개선이므로 항상 accept

$\Delta E > 0$ 확률적으로 개선 ($R <$)

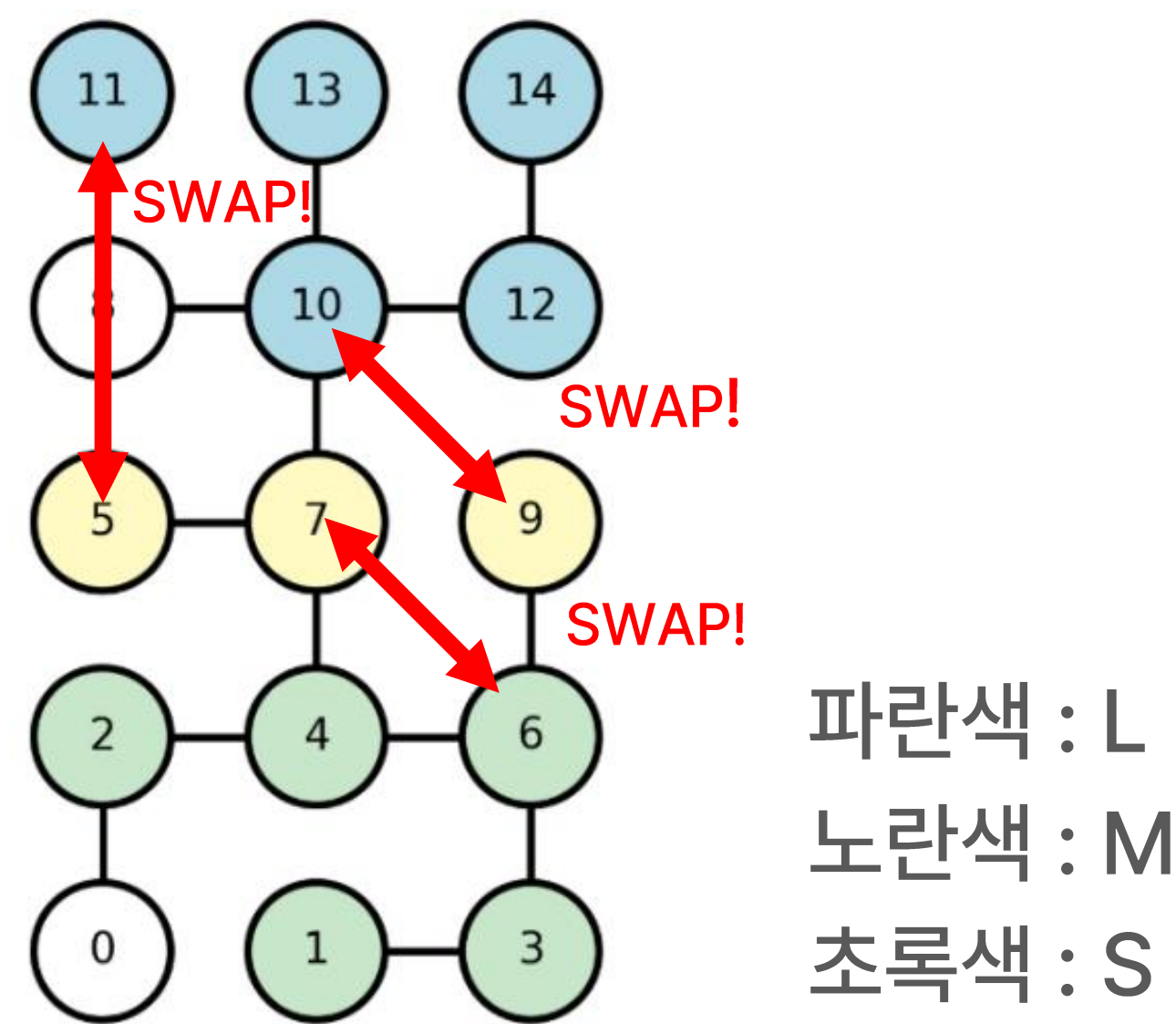
GA



파라미터 설정

파라미터	값
개체수	14
세대수	시간내 가능 세대수
Elite	4개
Mutation	60%
Termination	60Sec

ACO



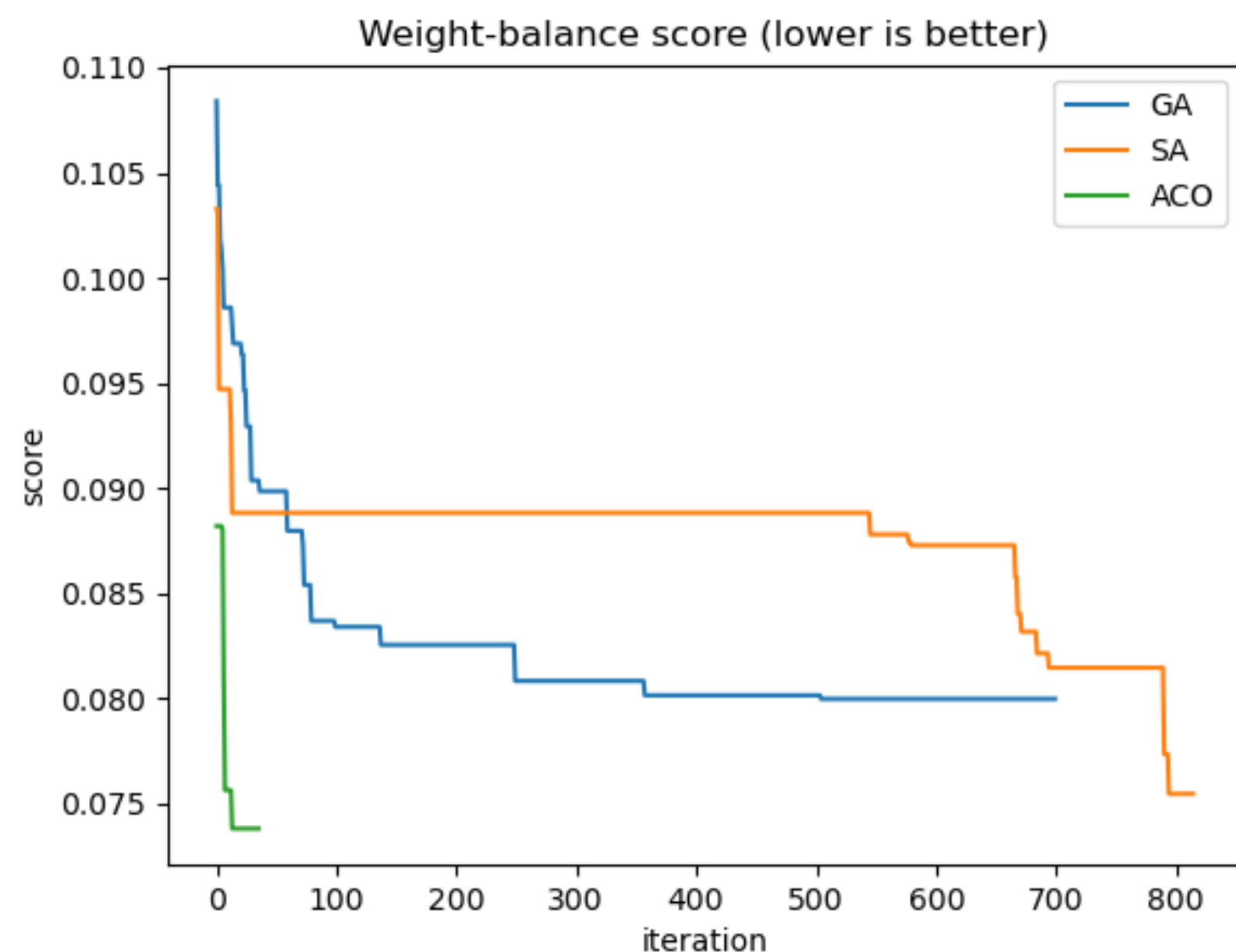
G = [..., [L,L,L,L,L,M,M,M,S,S,S,S,S], ...]

내부 리스트(g)를 하나 선택해 swap을 하고, 이후 fitness 값에 따라서 Ant colony system 형태로 페로몬 업데이트

파라미터 설정

파라미터	값
증발률	0.1
페로몬(alpha)	1.0
휴리스틱 지수(Beta)	2.0
Deposit 스케일	1.0
개미 수	10개
Step_per_ant	8개
후보 페어샘플	12개
Termination	60Sec

■ 메타휴리스틱 비교



1. 60초 동안 기존 알고리즘을 통해 Cost가 최소화된 해 탐색

2. 이후 각각의 메타휴리스틱을 통해 무게 밸런싱 최소화를

각 알고리즘별로 60초 동안 실행

SA는 빠른 속도로 새로운 해를 생성하나 해당 문제에서 수렴 속도가 느림

GA는 다른 두 비교군에 비해 속도, 해의 품질 측면에서 좋지 못한 결과를 보임

ACO는 초반에 해의 개선이 빠르게 이루어지나 속도 측면에서 부진한 결과를 보임

해당 문제에서 평균적으로 ACO가 비교군인 GA, SA보다 뛰어난 퍼포먼스를 보이는 것을 확인

■ 결론 및 시사점

- weight balancing 성능을 추가로 개선하는 메타휴리스틱 적용 가능성 확보하였다.
- 여러 메타휴리스틱을 비교군으로 삼아 좋은 알고리즘에 대한 기준을 확보
ACO가 해당 문제에서 뛰어난 성능을 보이며 ACO를 사용한다.
- 기존 MIP로 풀지 못하는 NP-Hard문제에 대해 확률적 컴퓨팅, 퍼지로지, 소프트 컴퓨팅을 활용하여 문제를 해결하였다.

■ 한계점 및 개선 방향

- 문제 인스턴스의 수요 구성과 공간 점유 구조에 따라 성능이 크게 달라질 수 있다.
- 주어진 기간 내 두가지를 한번에 계산하는 알고리즘을 구현하지 못하였다.
- 여러 휴리스틱의 파라미터를 조정하여 비교하는 실험의 부재로 추후 파라미터의 개선 후 비교군이 필요하며 보다 현실성을 반영할 수 있는 문제 구조로의 조정과 피트니스 함수 설계가 필요하다.